

AFENDA — FINAL TECHNOLOGY STACK SELECTION RECORD

**Version 2.0 · Approved implementation authority subordinate to
DOCTRINE.md**

AFENDA

Sources and release lines verified 7 August 2026

Executive approval

Architecture decision: APPROVED. No major technology replacement is recommended.

The selected stack is optimized for AFENDA's controlling constraint: **the coding agent must be able to generate, run, diagnose and repair the system through short, explicit, mechanically verifiable feedback loops.** Conventional enterprise strength does not qualify a technology if the primary development mode cannot handle it reliably.

Architecture quality assessment: approximately **9.7/10** against the stated selection criteria. This score evaluates the design of the stack, not whether the implementation is operationally proven. Evidence grades remain separate and may never be inflated by this approval.

Weighted assessment

Dimension	Assessment
Vibe-codeability and feedback loop	9.8/10
Financial integrity compatibility	9.8/10
Portability and self-hostability	9.7/10
Data/BI readiness	9.6/10
Security architecture	9.4/10
Operational simplicity at small-team scale	9.5/10
Escape-hatch reversibility	9.6/10
Current implementation evidence maturity	Not scored as architecture; tracked separately

1. Authority and scope

DOCTRINE.md governs what must be true. This record governs the approved tools and implementation shape used to make those truths executable.

Where the two conflict, the doctrine wins. A stack selection does not weaken a doctrine invariant. Exact patch versions live in machine-enforced files and lockfiles; this record selects release lines, contracts and qualification policy.

Selections are tagged:

- **[C] Constitutional stack boundary** — changes only on implementation contradiction, security/support event, failed qualification, incident or measured inability to meet an accepted requirement.
- **[D] Default implementation choice** — may change on written evidence that another option satisfies the same boundary with equal or better vibe-codeability and assurance.

Evidence grades use the doctrine scale. Most stack choices begin at **E1 Specified**. Historical results supplied by another implementation are marked with **-R** until independently reproduced.

2. Selection criteria

1. **SC-01 — Closed feedback loop.** Prefer failures caught by the compiler, schema validator or database constraint over silent runtime behavior.
2. **SC-02 — Stable API lineage.** Reject pre-stable or recently rewritten APIs on critical paths unless a small adapter and current docs fully contain the risk.
3. **SC-03 — Visible behavior.** Important behavior must be visible in the edited file or explicit dependency contract—not hidden in decorators, file location, proxying, runtime selection or implicit lifecycle.
4. **SC-04 — Small conceptual surface.** A small library with current docs in context is preferable to a broad framework with many sanctioned modes.
5. **SC-05 — Specific errors.** Prefer failures naming the violated type, schema, constraint, operation or control.

6. **SC-06 — Cheap local reproduction.** Authoritative behavior must be reproducible without a vendor control plane.
 7. **SC-07 — One sanctioned path.** Authoritative mutation, validation, routing, migration and posting each have one approved implementation path.
 8. **SC-08 — Provider portability.** The financial core targets plain PostgreSQL and standard protocols.
 9. **SC-09 — Operational restraint.** Add infrastructure only on measured need.
 10. **SC-10 — Executable selection.** A prose selection without a machine control is incomplete.
-

3. Canonical stack

Layer	Approved selection
Language	Strict TypeScript
Compiler	TypeScript 7 native + TypeScript 6 compatibility
Runtime	Node.js 24 LTS
Packages	pnpm workspaces + Turborepo
Database	PostgreSQL 18; compatibility floor 17
Driver	node-postgres
Query layer	Kysely + schema-derived codegen
Migrations	Reviewed forward-only SQL
API	Hono Node-only
Contracts	Zod 4 + REST/OpenAPI
Frontend	React 19.2 patched line + Vite 8.1
Frontend data	TanStack Router/Query/Table/Virtual + React Hook Form
UI	Tailwind CSS 4 + shadcn/ui
Identity	OIDC; Keycloak reference provider
Business authorization	AFENDA operation policies
Jobs	Graphile Worker behind adapter
Financial effects	Owned transactional outbox
Testing	Vitest + fast-check + StrykerJS + owned mutants
Database tests	PGLite fast + Testcontainers authoritative
E2E	Playwright
BI	reporting/analytics schemas + optional read-only Metabase
Observability	OpenTelemetry
Deployment	OCI + Docker Compose first

4. Detailed selection records

SEL-01 — Strict TypeScript as the single application language [C]

Vibe grade: A+

Current evidence: E1

Source basis: S01

Executable controls: SCC-01, SCC-02, SCC-03

Decision. Use strict TypeScript for domain, API, worker, transport schemas, metadata, generators, frontend, tests and governance. SQL remains the database language; configuration formats are not additional application languages.

Agent case. The compiler is the primary self-correction loop. A single application language maximizes useful model context and reduces duplicated concepts.

Engineering case. The browser is TypeScript regardless; using TypeScript on the backend removes a language instead of adding one. Shared concepts have one naming and testing culture.

Binding constraints. Enable strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes, useUnknownInCatchVariables, noImplicitReturns, noFallthroughCasesInSwitch, noPropertyAccessFromIndexSignature, verbatimModuleSyntax, isolatedModules and noEmit. Ban application any, unchecked domain casts and number-based authoritative money.

Revisit trigger. Only on measured evidence that TypeScript cannot meet an accepted requirement after SQL, worker-process and bounded-service mitigations, or when AI generation is no longer the primary implementation mode.

SEL-02 — TypeScript 7 native compiler plus TypeScript 6 compatibility tooling [D]

Vibe grade: A

Current evidence: E1

Source basis: S01

Executable controls: SCC-01, SCC-04

Decision. Use TypeScript 7 for the fast command-line type check and TypeScript 6 through @typescript/typescript6 for type-aware tooling until the TypeScript 7 programmatic API is qualified.

Agent case. TypeScript 7 materially shortens the red/green loop. The compatibility lane prevents speed from silently removing type-aware lint and ecosystem checks.

Engineering case. The official TypeScript transition supports side-by-side use. Compiler parity is more important than choosing one prematurely.

Binding constraints. Exact versions. Native and compatibility checks must both pass. Pin checker/builder parallelism; include a single-threaded parity lane. TypeScript or compiler upgrades are qualification events.

Revisit trigger. When TypeScript 7.1+ exposes a stable programmatic API and all required tooling is qualified without the compatibility compiler.

SEL-03 — Node.js 24 LTS runtime [C]

Vibe grade: A+

Current evidence: E1

Source basis: S02

Executable controls: SCC-04

Decision. Run API, workers, generators and governance on the exact patched Node.js 24 LTS baseline recorded in VERSION_BASELINE.json.

Agent case. Deep corpus, specific stack traces, broad library support and cheap local reproduction.

Engineering case. The Node project recommends LTS for production. Runtime consistency lowers self-host and support variance.

Binding constraints. Pin with .node-version or equivalent and assert process.version in CI. Do not adopt Current releases for production.

Revisit trigger. At a scheduled LTS migration or security/support lifecycle event, followed by full qualification.

SEL-04 — pnpm workspaces and Turborepo [D]

Vibe grade: A

Current evidence: E1

Source basis: S25

Executable controls: SCC-04, SCC-05

Decision. Use pnpm workspaces as package authority and Turborepo only for deterministic task orchestration and cacheable dependency ordering.

Agent case. One lockfile and explicit task graph give the agent a predictable repository loop.

Engineering case. Efficient workspace installs, exact dependency graph and simple monorepo execution without a second application framework.

Binding constraints. Exact packageManager field, frozen lockfile, no implicit install scripts unless allowlisted, no Nx/Bazel parallel authority.

Revisit trigger. Only when repository-scale evidence shows the task graph or workspace model cannot meet build and governance requirements.

SEL-05 — Plain PostgreSQL 18 with PostgreSQL 17 compatibility floor [C]

Vibe grade: A+

Current evidence: E3-R for ledger kernel; stack selection E1

Source basis: S03, S04

Executable controls: SCC-06, SCC-07

Decision. PostgreSQL is the permanent financial-system contract. Primary qualification runs on PostgreSQL 18; every declared database behavior also passes on PostgreSQL 17 until the floor is explicitly raised.

Agent case. SQL is visible, mature, locally reproducible and produces constraint-specific errors. Critical invariants can live below generated application code.

Engineering case. Deferred constraints, composite foreign keys, exact numeric, roles, RLS, SECURITY DEFINER, transactional outbox and reporting projections are native capabilities.

Binding constraints. No provider-only SQL or SDK in the financial core. Run migrations, ledger spine, role/grant, concurrency and restore lanes separately on both majors.

Revisit trigger. Never for the PostgreSQL contract; only the supported major floor and hosting profile change through qualification.

SEL-06 — node-postgres (pg) as canonical PostgreSQL driver [C]

Vibe grade: A

Current evidence: E1

Source basis: S06, S07

Executable controls: SCC-08, SCC-09

Decision. Use node-postgres Pool and checked-out Client beneath Kysely and direct SQL. Every transaction uses the same client instance from BEGIN through COMMIT or ROLLBACK.

Agent case. The driver is small, explicit and maps directly to PostgreSQL behavior. Transaction mistakes are easy to state and test.

Engineering case. No hidden transaction abstraction. Type parsers can keep bigint, numeric and high-precision values as strings for owned conversion.

Binding constraints. Never use `pool.query` inside a transaction. Own parsers for `bigint/numeric/date-time`. Parameterize data values and disallow dynamic identifiers outside reviewed helpers.

Revisit trigger. Only if a replacement proves equal PostgreSQL fidelity, smaller surface and stronger exact-type behavior under the full database suite.

SEL-07 — Kysely plus schema-derived code generation [C]

Vibe grade: A

Current evidence: E1

Source basis: S05

Executable controls: SCC-10

Decision. Use Kysely for ordinary typed queries; generate database types from the migrated PostgreSQL schema. Use reviewed SQL for ledger, migrations and advanced integrity operations.

Agent case. SQL-shaped code and compile-time schema drift provide a short, legible feedback loop.

Engineering case. The database remains the source of truth and raw SQL is first-class rather than an unsafe escape hatch.

Binding constraints. No hand-maintained database type authority. Codegen after migration. No Kysely plugin may hide tenant, transaction or money semantics.

Revisit trigger. If Kysely introduces breaking instability or cannot express a measured query need without unsafe loss of types; direct SQL remains the fallback.

SEL-08 — Reviewed forward-only SQL migrations [C]

Vibe grade: A+

Current evidence: E1

Source basis: S03, S04

Executable controls: SCC-07, SCC-11

Decision. Use checksummed numbered SQL migrations. Migrations are applied by a dedicated migration credential and verified before application code generation.

Agent case. The complete schema and constraint errors are visible. No migration DSL or competing schema language must be inferred.

Engineering case. Production changes remain reviewable, recoverable and independent of the query library.

Binding constraints. No schema push in shared or production environments. Destructive/data-transforming changes escalate. Test empty install, upgrade from prior release, realistic data, dual majors and restore.

Revisit trigger. Only if another SQL-first migration runner improves evidence without becoming a second schema authority.

SEL-09 — Hono as a Node-only thin HTTP adapter [D]

Vibe grade: A-

Current evidence: E1

Source basis: S08

Executable controls: SCC-12

Decision. Use Hono only through `@hono/node-server`. Route code parses transport, resolves actor/scope, calls one typed operation and maps Result to HTTP.

Agent case. Small visible API and no decorator/DI magic. Binding the runtime removes multi-runtime ambiguity.

Engineering case. The HTTP framework remains replaceable and contains no domain or persistence policy.

Binding constraints. Node adapter only. Default router only. No Hono RPC as authoritative client contract. No business logic, ad hoc SQL or transactions in route handlers.

Revisit trigger. On a breaking major or evidence that the thin-adapter boundary cannot be maintained.

SEL-10 — Zod 4 transport schemas and REST/OpenAPI contracts [C]

Vibe grade: A+

Current evidence: E1

Source basis: S09, S10

Executable controls: SCC-03, SCC-12

Decision. Use Zod 4 for every external boundary and @hono/zod-openapi as the sole API-schema path. Generate clients from OpenAPI; do not share server domain types over JSON.

Agent case. One schema produces runtime validation, static transport types and clear error paths.

Engineering case. Transport/domain separation preserves bigint money, rational rates and explicit instants across a lossy JSON boundary.

Binding constraints. Money minor units and rate components cross JSON as canonical integer strings. Validate request and response. Version authoritative endpoints. Hono RPC and hand-maintained duplicate client contracts are prohibited.

Revisit trigger. If the approved Zod/OpenAPI path becomes unstable or cannot express required standards without a second authority.

SEL-11 — React 19.2 patched line with Vite 8.1 SPA [C]

Vibe grade: A+

Current evidence: E1

Source basis: S11, S12, S13

Executable controls: SCC-14

Decision. Build a client-side React ERP delivered as static assets by Vite. Pin a patched React 19.2 line and prohibit all react-server-dom packages.

Agent case. The browser/server boundary is a literal HTTP call. There are no Server Components, Server Actions, framework cache modes or file-location execution rules to confuse.

Engineering case. An authenticated ERP gains little from SSR while static deployment is simple across managed and self-hosted profiles.

Binding constraints. No Next.js, React Server Components or Server Actions. Dependency graph rejects react-server-dom-*. Exact React patch lives in the lockfile.

Revisit trigger. Only if a measured product requirement needs server rendering and a new architecture can preserve explicit authoritative boundaries.

SEL-12 — TanStack Router, Query, Table and Virtual; React Hook Form [D]

Vibe grade: A-

Current evidence: E1

Source basis: S17

Executable controls: SCC-13

Decision. Use TanStack file-based routing only, TanStack Query for server state, Table/Virtual for dense grids, and React Hook Form integrated with the transport schemas.

Agent case. Typed route/search state and headless explicit components make assumptions visible.

Engineering case. ERP filters and drill-downs remain shareable URLs; grids remain license-safe and under design-system control.

Binding constraints. One routing mode only. Generated route tree is generated output and drift-checked. Do not use generic un-narrowed router types that degrade TypeScript performance.

Revisit trigger. If measured grid or route performance fails after documented narrowing and virtualization practices.

SEL-13 — Tailwind CSS 4 and shadcn/ui source-owned components [D]

Vibe grade: A+

Current evidence: E1

Source basis: S14

Executable controls: SCC-05

Decision. Use Tailwind CSS 4 and source-owned shadcn/ui components as the UI-system implementation.

Agent case. The actual component source is in the repository; the agent reads and edits behavior rather than guessing a remote component API.

Engineering case. No runtime design-system lock-in and full control of ERP density and accessibility.

Binding constraints. UI components obey owned design tokens, accessibility gates and Storybook/component contracts. Avoid arbitrary one-off class drift.

Revisit trigger. Only on measured maintainability, accessibility or performance failure.

SEL-14 — Standard OIDC boundary with Keycloak as reference provider [C] protocol / [D] provider

Vibe grade: A-

Current evidence: E1

Source basis: S15

Executable controls: SCC-15

Decision. Authentication integrates only through standard OIDC. Keycloak 26.7 is the reference self-hosted provider. AFENDA owns contextual business authorization.

Agent case. Identity-provider complexity is removed from generated application code; the integration surface is small and testable.

Engineering case. SSO, MFA, federation and directory integration remain in a specialized identity system, while amount/entity/period/maker-checker authority stays under ERP doctrine.

Binding constraints. No local password/auth framework in the core. Token validation and identity mapping are adapters. Keycloak roles do not replace AFENDA operation policies.

Revisit trigger. Provider may change only after OIDC conformance, self-host, security, migration and operational qualification. Protocol boundary remains.

SEL-15 — Graphile Worker behind an owned JobQueue adapter [D]

Vibe grade: B+

Current evidence: E1

Source basis: S16

Executable controls: SCC-16

Decision. Use Graphile Worker for generic at-least-once jobs such as email, PDFs, indexing and exports. Confine it behind one owned enqueue/worker adapter.

Agent case. The low-corpus library is reduced to a tiny local surface and current docs can be kept in context.

Engineering case. It reuses PostgreSQL and avoids early queue infrastructure. It is not the authority for financial effect state.

Binding constraints. Use public APIs only; never query private worker tables. Worker uses an appropriate direct/session-capable connection. Tasks are small, typed and idempotent.

Revisit trigger. When measured throughput, isolation, retention or operational needs exceed the qualified adapter.

SEL-16 — AFENDA-owned transactional outbox for financial/statutory effects [C]

Vibe grade: B

Current evidence: E1

Source basis: S16

Executable controls: SCC-17

Decision. Create financial and statutory side effects through an owned durable state machine committed in the same database transaction as the source fact.

Agent case. The state transition is explicit and testable rather than hidden in queue retry behavior.

Engineering case. Effectively-once business effect requires idempotency, acknowledgement, ambiguity handling and reconciliation beyond a generic job queue.

Binding constraints. Generic queue executes work; it never defines financial truth. External calls never occur inside the posting transaction. States, attempts, fingerprints and operator actions are persisted.

Revisit trigger. The implementation may move to another worker or workflow engine only if the owned state machine and evidence remain unchanged.

SEL-17 — S3-compatible object storage and PostgreSQL search first [D]

Vibe grade: A-

Current evidence: E1

Source basis: S03

Executable controls: SCC-05

Decision. Use an owned S3-compatible storage port for documents and PostgreSQL full-text search plus pg_trgm for initial search.

Agent case. Small explicit adapters and familiar SQL avoid early infrastructure complexity.

Engineering case. Database stores immutable object metadata and digests; search remains a rebuildable projection.

Binding constraints. No object-store vendor types outside the adapter. No OpenSearch or cache until measurements justify it. Search never becomes financial authority.

Revisit trigger. On measured volume, linguistic or independent-scaling requirements.

SEL-18 — OpenTelemetry-first observability [C]

Vibe grade: A-

Current evidence: E1

Source basis: S24

Executable controls: SCC-05

Decision. Instrument API, worker and browser with OpenTelemetry; export through an OpenTelemetry Collector to chosen backends.

Agent case. One standard correlation model and specific runtime evidence improve agent diagnosis.

Engineering case. Backend vendor remains replaceable while traces, metrics and structured operation attributes are consistent.

Binding constraints. Initialize instrumentation before application code. Correlate actor, tenant/entity, operation, source and batch IDs without leaking money, personal data or secrets.

Revisit trigger. Only if the standard cannot meet a qualified operational requirement.

SEL-19 — Vitest, fast-check, StrykerJS and owned domain mutants [C]

Vibe grade: A

Current evidence: E1 for stack; historical ledger evidence E3-R

Source basis: S20, S21, S22

Executable controls: SCC-18

Decision. Use Vitest for the main test loop, fast-check for properties, StrykerJS for broad mutation and an independently maintained corpus of domain mutants for critical ERP invariants.

Agent case. Fast, specific failures enable self-correction; properties and mutants expose plausible but wrong generated code.

Engineering case. Generic mutation score is secondary. Critical domain faults—money, scope, idempotency, time, locks, outbox and reversal—must have zero survival.

Binding constraints. Tests may not be weakened without Reviewed approval. Critical mutant survival zero. Equivalent mutants require retained proof. Incident defects become permanent regression mutants.

Revisit trigger. Tool may change only if fault sensitivity and evidence quality are preserved or improved.

SEL-20 — PGLite fast lane and Testcontainers PostgreSQL authoritative lane [C]

Vibe grade: A

Current evidence: E1

Source basis: S18, S19

Executable controls: SCC-07, SCC-19

Decision. Use PGLite for fast deterministic SQL feedback and Testcontainers with real PostgreSQL for roles, concurrency, isolation, recovery and supported-major qualification.

Agent case. Fast local checks preserve the generation loop; the server lane prevents WASM convenience from becoming false assurance.

Engineering case. PGLite is single-connection; real role topology, multi-session races and restore behavior require actual PostgreSQL.

Binding constraints. Concurrency-tagged tests cannot run only on PGLite. Testcontainers images are digest-pinned. Both PostgreSQL 18 and 17 lanes are independent.

Revisit trigger. Only if another local/server pair gives equal speed and stronger PostgreSQL fidelity.

SEL-21 — Playwright for end-to-end ERP workflows [D]

Vibe grade: A

Current evidence: E1

Source basis: S20

Executable controls: SCC-20

Decision. Use Playwright for browser qualification of the complete vertical slice and critical operator workflows.

Agent case. Browser traces, screenshots and explicit actions provide a reproducible visible failure.

Engineering case. The system must prove API, UI, database and audit behavior together, not only isolated functions.

Binding constraints. At minimum: create invoice, approve, post, reverse, rebuild/view trial balance and inspect audit evidence. Trace retained on failure.

Revisit trigger. Only if a replacement provides equal cross-browser reliability and debugging evidence.

SEL-22 — Governed reporting schemas, read-only Metabase and CDC readiness [C] model / [D] tool

Vibe grade: A

Current evidence: E1

Source basis: S23, S03

Executable controls: SCC-21, SCC-22

Decision. Use afenda for authoritative data, reporting for governed projections and analytics for BI marts. Metabase is the optional read-only self-host BI profile. Design for CDC but create a slot only with a live monitored consumer.

Agent case. Clear schema boundaries and SQL projections are visible and locally testable.

Engineering case. BI is language-independent. Core dimensions use entity-scoped columns/FKs; analytical extension dimensions use normalized bridges; JSONB remains opaque metadata.

Binding constraints. BI role has read-only schema access. No posting EXECUTE. CDC slot requires owner, consumer, lag monitoring, WAL budget, snapshot and removal procedure.

Revisit trigger. BI tool may change; reporting schema contract and CDC safety remain.

SEL-23 — OCI containers and Docker Compose first [C]

Vibe grade: A

Current evidence: E1

Source basis: S02, S03

Executable controls: SCC-23

Decision. Ship web, api, worker, PostgreSQL, Keycloak, optional object storage, Metabase and OpenTelemetry Collector as digest-pinned OCI profiles; Docker Compose is the first self-host profile.

Agent case. The full failure surface is reproducible locally without a cloud control plane.

Engineering case. Self-hostability and provider portability are product requirements; Kubernetes is a later deployment profile, not an application dependency.

Binding constraints. After dependency and image acquisition, verification runs with external egress denied. Fresh-airgapped installation is not claimed without an artifact mirror. Helm later only after demand.

Revisit trigger. Add HA/Kubernetes profiles on customer/SLA evidence, not fashion.

SEL-24 — Modular monolith in one repository [C]

Vibe grade: A+

Current evidence: E1

Source basis: S05

Executable controls: SCC-05, SCC-24

Decision. Build one modular application with deployable api and worker processes. Domain modules expose commands, queries, events and read contracts—not repositories, tables or internals.

Agent case. The agent can inspect end-to-end behavior without distributed-service context loss.

Engineering case. Strong boundaries without the operational cost, partial failure modes and data ownership complexity of premature microservices.

Binding constraints. Feature-first packages, root public APIs, dependency graph checks, no cross-module data writes, no transport concerns in domain packages.

Revisit trigger. Extract a service only on measured scaling, isolation, deployment or ownership evidence with a stable contract.

SEL-25 — Functions, immutable data and explicit capability objects [C]

Vibe grade: A+

Current evidence: E1

Source basis: S01

Executable controls: SCC-24

Decision. Domain/application operations use functions, immutable data and explicit dependency objects. Classes are allowed only at adapters where a library requires them and behavior remains visible.

Agent case. An operation is understandable from its input, dependencies, control flow, result and tests.

Engineering case. Avoids hidden proxy transactions, decorator DI, inheritance state and service-locator coupling.

Binding constraints. Forbidden: class-based domain inheritance, decorator-driven DI, reflection, service locators, implicit transactions, ambient global state and runtime module discovery.

Revisit trigger. Only if a pattern proves equally visible and mechanically enforceable.

SEL-26 — AI as proposal-only capability [C]

Vibe grade: A

Current evidence: E1

Source basis: S10

Executable controls: SCC-25

Decision. AI may extract, classify, explain, match and propose structured changes. It never holds a ledger capability and cannot create an authoritative financial fact.

Agent case. Structured Zod outputs keep AI behavior inside the same TypeScript feedback and validation system.

Engineering case. Model uncertainty, provider change and prompt behavior remain outside the accounting authority. Human-approved operations own posting.

Binding constraints. Store model/prompt/input provenance and confidence. AI writes proposals only. No autonomous posting, payment release, period close or statutory submission.

Revisit trigger. Automation may increase only after qualified accuracy, control, audit and regulatory evidence; ledger authority remains human-governed.

SEL-27 — Declarative/trusted extensions first; untrusted code out of process [C]

Vibe grade: B

Current evidence: E1

Source basis: S03

Executable controls: SCC-26

Decision. Phase-one extensions are declarative fields, view slots, document types, trusted version-controlled policies and after-commit hooks. Untrusted executable code runs only in a separate isolated process/service with capability-scoped APIs.

Agent case. Avoids low-corpus, security-sensitive in-process sandbox semantics.

Engineering case. The operating-system/process boundary is more auditable than a clever JavaScript isolate with direct database reach.

Binding constraints. No direct database credential for untrusted extensions. No execution inside posting. Time, memory, network, data and failure scopes are declared.

Revisit trigger. Only after a separately qualified extension-host design and threat model.

5. Explicit rejections

Rejected option	Reason
Java/Spring or C# backend	Strong conventional engineering but violates the one-language, fast-feedback, vibe-code-primary constraint.
Next.js, React Server Components and Server Actions	Invisible server/client and cache semantics on authoritative code; RSC dependency is explicitly prohibited.
NestJS and decorator-heavy dependency injection	Hidden provider resolution, lifecycle and reflection.
Drizzle before a settled stable 1.x	API-churn risk under the project's version-mixing criterion.
Prisma as database authority	Second schema language and migration authority outside reviewed SQL.
GraphQL for authoritative commands	Weakens explicit operation identity, idempotency and audit semantics.
Bun or Deno as primary runtime	Less stable primary corpus and additional runtime-specific behavior.
Microservices, Kafka, Redis or Temporal on day one	Unjustified operational and failure surface.
Vendor-specific ledger primitives	Breaks plain PostgreSQL portability and self-host qualification.
Direct browser-to-database access	Bypasses operation contracts, evidence, idempotency and authorization.
In-process untrusted extension sandbox	Security boundary is too subtle for the project's generation model.
Runtime-editable executable business scripts	Unversioned behavior outside code review and verification.
AI autonomous posting	Model output cannot become authoritative financial substance.

6. Escape hatches

ESC-01 — CPU-bound workloads

Use, in order: **set-based SQL**, then **Node worker processes/worker threads**, then a **bounded specialist service** only after profiling proves CPU—not database or I/O—is the limiting factor. The specialist service owns no ledger credential and returns a proposal/result through a narrow versioned contract.

ESC-02 — Concurrency

Concurrency remains an Owned surface. Posting, period close, numbering, idempotency, valuation and restatement coordination live in one small package. Nothing passes without real multi-session PostgreSQL tests, killed lock-order/isolation mutants, whole-transaction retry evidence and an independent final-state oracle.

ESC-03 — Untrusted extensions

Do not build an in-process sandbox initially. Untrusted code runs in a separate killable process/service with no database credential and a capability-scoped API. Phase-one extensions remain declarative or trusted/version-controlled.

ESC-04 — Future service extraction

A module may become a service only after a measured scaling, isolation, deployment or ownership requirement exists and the stable contract, data ownership, failure handling and reconciliation are already executable in the monolith.

7. Executable selection controls

Control	Name	Applies to	Required outcome
SCC-01	TypeScript native/compat parity	SEL-01, SEL-02	TypeScript 7 native check, TypeScript 6 stable-order compatibility check, and type-aware lint all pass.
SCC-02	Strictness and no-any	SEL-01	Strict compiler options enabled; application any and unchecked casts are rejected.
SCC-03	Authoritative money type safety	SEL-01, SEL-10	No JavaScript number in authoritative money contracts or code paths; JSON money uses canonical integer strings.
SCC-04	Exact toolchain pins	SEL-02, SEL-03, SEL-04	Runtime, compilers, package manager, dependencies and lockfile are exact and CI-verified.
SCC-05	Module boundaries	SEL-24, SEL-25	Dependency graph forbids domain-to-adaptor imports, internal subpaths and cycles.
SCC-06	Provider-neutral financial core	SEL-05	No managed-vendor SDK or provider-only SQL primitive in ledger and database kernel paths.
SCC-07	Dual PostgreSQL major qualification	SEL-05, SEL-08, SEL-20	Migrations and authoritative database controls pass separately on PostgreSQL 18 and PostgreSQL 17.
SCC-08	Single-client transaction rule	SEL-06	Every transaction uses one checked-out node-postgres client from BEGIN through COMMIT/ROLLBACK.
SCC-09	Exact database type parsing	SEL-06	bigint/numeric/date-time authoritative values are parsed under owned rules, not implicit JavaScript coercion.

Control	Name	Applies to	Required outcome
SCC-10	Schema-derived Kysely types	SEL-07	Kysely types regenerate from migrated PostgreSQL; drift fails CI.
SCC-11	Migration integrity	SEL-08	Migration checksums, ordering and review status are verified; no schema push in shared or production environments.
SCC-12	Hono runtime and contract path	SEL-09, SEL-10	Only @hono/node-server and @hono/zod-openapi appear in authoritative API code; Hono RPC is prohibited.
SCC-13	One frontend routing mode	SEL-12	TanStack file-based routing only; generated route tree drift fails CI.
SCC-14	No React Server Components	SEL-11	No react-server-dom-* dependency and no Server Components/Server Actions architecture.
SCC-15	OIDC boundary	SEL-14	Authentication uses standard OIDC; contextual ERP authorization remains inside AFENDA operations.
SCC-16	Queue adapter boundary	SEL-15	Graphile Worker appears only behind the owned JobQueue adapter; private worker tables are not queried.
SCC-17	Financial outbox required	SEL-16	Every authoritative external financial/statutory effect is created through the owned outbox state machine in the source transaction.
SCC-18	Critical mutation survival zero	SEL-19	All critical TypeScript and domain mutants are killed; exceptions require time-boxed Owned approval.
SCC-19	Concurrency lane enforcement	SEL-20	Concurrency-tagged tests are rejected on PGLite and run on real multi-session PostgreSQL.
SCC-20	Browser workflow gate	SEL-21	Playwright verifies the vertical ERP flow and audit evidence before release.
SCC-21	Read-only BI boundary	SEL-22	BI role has SELECT only on reporting/analytics schemas and no posting-function execution.
SCC-22	CDC slot safety	SEL-22	No logical replication slot exists without a named live consumer, lag alert, WAL budget and removal procedure.
SCC-23	Reproducible self-host profile	SEL-23	After lockfile dependencies and digest-pinned images are acquired, the stack and verification run with external egress denied.
SCC-24	Explicit application architecture	SEL-24, SEL-25	No decorator DI, service locator, class-based domain inheritance, implicit transaction or hidden runtime module discovery.
SCC-25	AI cannot post	SEL-26	AI identities and proposal workers cannot call or insert into the ledger; human-approved operation path remains mandatory.
SCC-26	Extension isolation	SEL-27	Untrusted executable extensions have no in-process execution and no direct database credential.

Control	Name	Applies to	Required outcome
SCC-27	Stack authority integrity	All	STACK.md hash matches STACK.sha256 and every selection/control has complete metadata.

8. Required repository architecture

```

/
├─ apps/
│   ├── web/          # React + Vite
│   ├── api/          # Hono Node adapter
│   └─ worker/        # Graphile Worker adapter + outbox executor
├─ packages/
│   ├── domain/        # pure types, value objects and rules
│   ├── operations/    # commands and queries
│   ├── contracts/     # Zod + OpenAPI transport schemas
│   ├── db/            # Kysely types and transaction adapters
│   ├── errors/        # canonical Result/failure contracts
│   ├── authz/         # operation authorization
│   ├── money/         # exact money, FX and rounding boundaries
│   ├── time/          # clocks and temporal policies
│   ├── testing/       # fixtures, oracles and mutant support
│   └─ ui/             # source-owned UI system
├─ db/
│   ├── migrations/
│   ├── functions/
│   ├── verification/
│   └─ mutants/
├─ doctrine/
├─ stack/
├─ scripts/
├─ pnpm-workspace.yaml
├─ turbo.json
└─ tsconfig.base.json

```

Application flow

Hono route

- parse Zod/OpenAPI transport
- resolve authenticated actor and entity scope
- call one typed operation with explicit capabilities
- execute one explicit node-postgres/Kysely transaction when needed
- return canonical Result mapped to HTTP

Route handlers, UI components and workers do not contain authoritative business policy or ad hoc ledger SQL.

9. Required gates

Gate	Required contents	Trigger
Fast	TypeScript native check; TypeScript 6 compatibility check; type-aware lint; module boundaries; affected Vitest/property tests; lightweight PGLite checks	Every commit
Merge	PostgreSQL 18 and 17 affected migrations/tests; role/grant and posting controls; affected Stryker/domain mutants; API contract diff; build	Every pull request
Nightly	Full domain mutation corpus; multi-session concurrency; projection rebuild/reconciliation; full Playwright critical paths	Scheduled
Qualification	Clean install from lockfile; all SCC controls; both PostgreSQL majors; migration upgrade; credential topology; restore; exact version/digest evidence	Release and toolchain/platform change
Production	Reconciliation, privilege/scope drift, outbox ambiguity, backup/restore cadence, incident regressions, operational objectives	Governed cadence

A routinely bypassed or waited-out gate is a defect in its cadence. Split or relocate the gate; do not silently remove the control.

10. Version and pinning policy

- Release lines are selected here; exact patches and image digests are machine authority.
- Agents may not upgrade Node, TypeScript, PostgreSQL, React, Vite, Keycloak or constitutional dependencies.
- A runtime/compiler/database major or qualified toolchain change requires representative regeneration, semantic diff, full applicable controls and explicit approval.
- Security patch upgrades are expedited but still run affected qualification.
- `package.json`, `pnpm-lock.yaml`, `.node-version`, container digests and CI image constants must agree with `VERSION_BASELINE.json`.
- No unbounded `latest`, `caret` or `tilde` range is allowed for constitutional dependencies in the committed application manifest.

11. Current evidence ledger

This approval concerns architecture. It does not convert historical or supplied results into independent proof.

Area	Honest starting grade
Stack architecture and selection rationale	E1 — Specified
Ledger controls previously reported with mutation evidence	E3-R — Reported mutation-sensitive until clean reproduction
SQL migrations and test infrastructure	E1 unless fresh sealed results are attached
Real PostgreSQL concurrency, restore and credential topology	Not E5 until qualification
Deployment operations	Not E6; “battle-proven” remains prohibited

Evidence advances in validation records, not by rewriting the selection rationale.

12. Adoption and freeze

This record is ready for adoption after the checklist in `ADOPTION_CHECKLIST.md` is completed and `STACK.sha256` is committed.

After adoption, stack changes require one of:

- a failed qualification lane;
- surviving critical mutant;
- implementation contradiction;
- security advisory or supported-version lifecycle event;
- measured performance or operational failure;
- doctrine conflict;
- a revisit trigger recorded on the affected selection.

Fashion, a newer release, additional theoretical review or another vendor comparison is not evidence.

13. Source register

- **[S01] TypeScript Team, *Announcing TypeScript 7.0*.** TypeScript 7 GA, 8–12x typical full-build speedups, TypeScript 6 compatibility package, no stable 7.0 programmatic API.
<https://devblogs.microsoft.com/typescript/announcing-typescript-7-0/>
- **[S02] Node.js Project, *Node.js Releases*.** Node 24 is LTS; production should use Active or Maintenance LTS.
<https://nodejs.org/en/about/previous-releases>
- **[S03] PostgreSQL Global Development Group, *PostgreSQL 18 Released*.** PostgreSQL 18 is the current major release.
<https://www.postgresql.org/about/news/postgresql-18-released-3142/>
- **[S04] PostgreSQL Global Development Group, *PostgreSQL 18.4 and 17.10 Released*.** Current supported patch baselines used in this record.
<https://www.postgresql.org/about/news/postgresql-184-1710-1614-1518-and-1423-released-3297/>
- **[S05] Kysely, *Kysely documentation*.** Thin SQL-shaped query builder, database-derived types, one-to-one compilation, zero dependencies.
<https://kysely.dev/>
- **[S06] node-postgres, *Transactions*.** All transaction statements must use the same checked-out client; `pool.query` is not safe for transactions.
<https://node-postgres.com/features/transactions>
- **[S07] node-postgres, *Data types and parsing*.** Exact and custom type parsing; values can remain strings rather than being silently coerced.
<https://node-postgres.com/features/types>
- **[S08] Hono, *Node.js adapter*.** Official Node adapter and server pattern.
<https://hono.dev/docs/getting-started/nodejs>
- **[S09] Hono, *Zod OpenAPI example*.** Zod validation and OpenAPI generation through one approved path.
<https://hono.dev/examples/zod-openapi>
- **[S10] Zod, *Zod 4 documentation*.** Zod 4 is stable and maps closely to TypeScript types.
<https://zod.dev/>
- **[S11] Vite, *Vite 8.1 announcement*.** Vite 8.1 stable release line.
<https://vite.dev/blog/announcing-vite8-1>
- **[S12] React Team, *React 19.2*.** React 19.2 release line.
<https://react.dev/blog/2025/10/01/react-19-2>

- **[S13] React Team, *RSC security fixes*.** React Server Components fixes through 19.2.4; this stack prohibits react-server-dom packages.
<https://react.dev/blog/2025/12/11/denial-of-service-and-source-code-exposure-in-react-server-components>
- **[S14] Tailwind Labs, *Tailwind CSS v4.3*.** Tailwind CSS v4 current release family.
<https://tailwindcss.com/blog>
- **[S15] Keycloak, *Keycloak 26.7.0 released*.** Current reference self-hosted OIDC provider line.
<https://www.keycloak.org/2026/07/keycloak-2670-released>
- **[S16] Graphile Worker, *Graphile Worker documentation*.** PostgreSQL-backed, at-least-once background jobs with retries; small-team infrastructure simplification.
<https://worker.graphile.org/docs>
- **[S17] TanStack, *Router type safety*.** Type-safe routes, params and search state; runtime mismatch detection.
<https://tanstack.com/router/latest/docs/guide/type-safety>
- **[S18] Testcontainers, *PostgreSQL module for Node.js*.** Real disposable PostgreSQL for authoritative integration tests.
<https://node.testcontainers.org/modules/postgresql/>
- **[S19] PGLite, *Getting started*.** Embedded PostgreSQL in WASM; single exclusive connection means concurrency is not qualified there.
<https://pglite.dev/docs/>
- **[S20] Vitest, *Vitest CLI*.** Fast TypeScript/ESM test loop.
<https://vitest.dev/guide/cli>
- **[S21] fast-check, *Introduction*.** Property-based test generation and shrinking.
<https://fast-check.dev/docs/introduction/>
- **[S22] Stryker, *StrykerJS introduction*.** Mutation testing for JavaScript and TypeScript.
<https://stryker-mutator.io/docs/stryker-js/introduction/>
- **[S23] Metabase, *PostgreSQL connection*.** Read-only schema-scoped PostgreSQL BI connection.
<https://www.metabase.com/docs/latest/databases/connections/postgresql>
- **[S24] OpenTelemetry, *JavaScript getting started*.** Node.js and browser telemetry instrumentation.
<https://opentelemetry.io/docs/languages/js/getting-started/>
- **[S25] pnpm, *pnpm documentation*.** Workspace support, exact lockfile workflow, monorepo package management.
<https://pnpm.io/>